

Sadržaj

1. PROGRAMSKI JEZICI	2
1.1. Razvoj programskih jezika	2
2. PRIKAZ DELPHI JEZIKA	4
2.1. Putevi razvoja.....	4
2.1.1. GLAVNE KARAKTERISITKE DELPHIJA 1	4
2.1.2. GLAVNE KARAKTERISITKE DELPHIJA 2	4
2.1.3. GLAVNE KARAKTERISITKE DELPHIJA 3	5
2.1.4. GLAVNE KARAKTERISITKE DELPHIJA 4	5
2.1.5. GLAVNE KARAKTERISITKE DELPHIJA 5	6
3. STRUKTURA DELPHIJA.....	6
3.1. Elementi Delphija	6
3.2. Delphi kao alatka za vizuelno programiranje	8
3.3. Datoteke koje Delphi generiše.....	9
4. BAZE PODATAKA.....	11
4.1. Potreba za standardom pri radu sa bazama podataka.....	11
4.2. Terminologija baza podataka.....	11
4.3. Komponente za rad sa bazama podataka	13
4.3.1. TABLE I QUERY.....	15
4.3.1.1. Kretanje kroz tabelu.....	17
4.3.1.2. Određivanje sadržaja polja	17
4.3.1.3. Pretraživanje tabele.....	17
4.3.1.4. Promena vrednosti polja	18
4.3.1.5. Master - detail relacija	19
4.3.2. FIELD KOMPONENTA.....	20
4.3.3. DATABASE GRID.....	21
4.4. Pravljenje programa za rad sa bazama podataka	21
4.5. Lokalne i client / server aplikacije	22
4.6. Borland database engine (BDE).....	23
5. OPIS PROBLEMA.....	25
5.1. ZADATAK	25
5.2. OBJAŠNJENJE STRUKTURE PROGRAMA	25
5.2.1. FORMA - UNOS ARTIKLA.....	26
5.2.2. FORMA ZA UNOS ULAZNOG NALOGA	28
5.2.3. FORMA ZA PREGLED ZADUŽENJA.....	30
6. ŠTA JE POTREBNO ZA DISTRIBUIRANJE APLIKACIJE?.....	33
7. ZAKLJUČAK	34
8. Literatura	35

1. PROGRAMSKI JEZICI

1.1. Razvoj programskih jezika

Programski jezik predstavlja notaciju namenjenu izražavanju algoritma na način koji može razumeti i izvršiti računar. To je skup simbola sa zadatim pravilima formiranja od tih simbola pravilnih konstrukcija programskih jezika.

Danas postoji nekoliko stotina različitih programskih jezika. Dosadašnji razvoj programskih jezika možemo podeliti na sledeći način :

1. Mašinski jezici
2. Simbolički mašinski jezici
3. Proceduralni programski jezici
4. Neproceduralni programski jezici
5. Objektno orjentisani jezici
6. Jezici veštačke inteligencije

Najelementarniji programski jezik je interni jezik računara, tzv. mašinski jezik. On je izgrađen nad binarnom azbukom. Svaka mašinska naredba predstavlja binarni kod koji računar može da prepozna i izvrši.

Simbolički mašinski jezik je nastao iz mašinskog zamenom koda operacije simboličkim imenima, još se zove i mašinski orjentisan jezik ili assembler.

Proceduralni programski jezici omogućavaju opis algoritma na mnogo prirodniji i jednostavniji način u poređenju sa simboličkim mašinskim jezicima. To su jezici nezavisni od mašine i način zadavanja algoritma je blizak načinu izražavanja ljudi.

Neproceduralni programski jezici omogućavaju pisanje programa na znatno većem nivou apstrakcije, težeći ka “opisnim” formalizacijama algoritama proširenjem standardnih operatora kompleksnim operatorima. Ovi jezici specificiraju šta treba uraditi umesto zadavanja kako to treba uraditi, pa se nazivaju i jezici vrlo visokog nivoa.

Objektno orjentisani jezici zasnivaju se na objektima. Objekat ili klasa sadrže podatke i operacije koje se vrše nad tim podacima. Do sada su, u prethodnim opisima programskih jezika, podaci i operacije koje se vrše nad tim podacima bili razdvojeni. Objekat je sličan strukturi u C-u ili recordu u Pascalu, ali sa dodatkom procedura i funkcija koje “obrađuju” definisane podatke. Ideja je da se napravi klasa koja nešto radi i da mi tu klasu možemo koristiti u svojim programima, ali da neznamo kako ona radi i bez mogućnosti da utičemo na to.

Na primer: Imamo klasu *Mojstring* koja ima promenljivu *tekst* i funkcije *VratiDužinuTeksta*, *PretvoriUVelikaslova*, *PretvoriUMalaslova*. (sledi definicija klase u Delphiju)

```
type
  TMojString = class(TString)
  private // Private declarations
  // ovde je def. privatnih procedura i prom. koje ne možemo koristiti van
  klase
```

```

public    // Public declarations–sve prom. definisane kao public text :
string;
    // možemo koristiti van klase
function VратиДуžинуТекста : integer;
procedure PretvoriUVelikaSlova;
procedure PretvoriUMalaSlova;
end;
// pa sada sledi implementacija svih funkcija i procedura
...
// u nekom drugom programu
var s : MojString;
n : integer;
...

```

s je objekt tipa MojString i možemo pisati

```

...
s.Tekst:= 'Diplomski rad';
n:=s.VратиДуžинуNiza;    // n=13;
s.PretvoriUVelikaSlova;  // vrednost s.tekst= 'DIPLOMSKI RAD';
s.PretvoriUMalaSlova;    // vrednost s.tekst= 'diplomski rad';
...

```

Znači koristimo funkciju *VратиДуžинуNiza* i proceduru *PretvoriUVelikaSlova* a da ne znamo kako one rade. Bitan nam je samo rezultat.

Jezici veštačke inteligencije su u fazi razvoja. Osnovu za ove jezike čine funkcionalni jezik lisp, logički jezik prolog i drugi jezici veštačke inteligencije. To su jezici koji će omogućiti predstavljanje znanja i deduktivno zaključivanje na tom znanju.

2. PRIKAZ DELPHI JEZIKA

2.1. Putevi razvoja

Postoje već pet generacije Delphija; sa svakim novim izdanjem proizvod je postajao sve savršeniji. Zbog mnogobrojnih izmena koje je Delphi pretrpeo, neke od starijih aplikacija i komponenata mogu zahtevati neznatne izmenene koda da bi proradile u novim verzijama. Jedan od razloga zbog kojih biste mogli da jednostavno ažurirati aplikacije Delphija jeste taj što su klase objektnih interfejsa unapređene, ali ne i bitno izmenjene. implementacija objekata je obnovljena, na nekim mestima drastično, ali kako su njihove klase interfejsa neizmenjene, programi koji ih koriste samo moraju biti ponovo prevedeni kako bi se prebacili u oblik koji može dalje da se razvija pomoću nove verzije Delphija.

2.1.1. Glavne karakteristike Delphija 1

Prva verzija Delphija izdata otprilike u isto vreme kad i Windows 95, predstavljala je prvo okruženje za vizuelno programiranje koje prevodi program u pravi, mašinski kod. Iako je Visual Basic već postojao, u njemu se stvarao kod koji je radio pod kontrolom prevodioca. Delphi prevodi tekst izvornog koda direktno u naredbe mašinskog koda.

Karakteristike :

- Paleta komponenata za vizuelne komponente koje mogu biti ubačene u forme (Svaka komponenta je i dalje obekat).
- Prozor Object Inspektora.
- Kartica *Properties* – koristi se za dodeljivanje vrednosti svojstava komponenata.
- Kartica *Events* – ubacuje procedure za rad sa događajima za takve objekte kao što je procedura koja se poziva kada korisnik pritisne dugme.
- Automatsko stvaranje koda – Delphi stvara deklaracije za klase i promenljive, a i procedure za rad sa događajima.
- Datoteke sa mašinskim .exe kodom – Prevedene aplikacije sadrže potpunu biblioteku vizuelnih komponenata, što ih delimično čini potrošačima slobodnog prostora na disku, ali veoma brzim.
- Podrška kontrolama Visual Basica (VBX kontrolama).
- Programiranje baza podataka pomoću vizelnih komponenata i Borland Database Enginea (BDE) – Mogućnost pisanja potpuno operativnih aplikacija za rad sa bazama podataka verovatno je i glavni razlog izuzetnog uspeha Delphija.

2.1.2. Glavne karakteristike Delphija 2

- Duge i kratke znakovne nizove .

- 32 – bitni ceo broj.
- Dodeljivanje memorije ograničeno je isključivo količinom slobodne memorije koja može iznositi i više megabajta.
- Izvršavanje više niti programa.
- Potpuna podrška 32 – bitnim kontrolama Windowsa.
- Unapređena podrška za povezivanje i ugrađivanje objekata (engl. *Object Linking and Embedding – OLE*).
- Zahteva Windows 95 ili NT.
- Unapređen BDE i SQL (struktuirani jezik upita; engl. *Structured Query Language*) veze.

2.1.3. Glavne karakteristike Delphija 3

- Podrškama paketima koda – sastoji se u tome što omogućava da više aplikacija dele kod. Dobit je u tome što u ovom slučaju više nije neophodno da svaka od prevedih aplikacija sadrži celokupan prevedeni VCL (Visual Components Library), kao što je bio slučaj u prethodne dve verzije.
- Brojna unapređenja VCL-a – uključujući nove Win32 komponente.
- Potpuna podrška ActiveX kontrolama, uključujući i njihovu izradu.
- Mogućnost prevođenja datoteka sa programskim modulima u datoteke objektnog koda (.obj) – Ovo omogućava prevedenim modulima Delphija da budu uvezani u druge sisteme za izradu softvera kao što je C++.
- Internet i intranet komponente – komponente za komunikaciju sa Web serverom.
- Podrška tipu znakovnih nizova WideString.
- Standardna, profesionalna i klijentsko-serverksa izdanja.

2.1.4. Glavne karakteristike Delphija 4

- Dinamički nizovi, preklapanje metoda, novi 64-bitni tip podataka Integer...
- Project Manager – Stari je potpuno preuređen u novi model koji podseca na drvo, što omogućava lakše praćenje podataka.
- Module Explorer – U obliku drveta prikaz svih procedura, promenljivih i komponenti koje trenutno koristimo u programu i lak skok sa deklaracije na implementaciju procedura.
- Spojivi prozori sa alatima – Delphijevi prozori sa alatima se mogu “privezati” za glavni prozor ili “odvezati” i egzistirati kao samostalni prozori.
- Lakše otklanjanje grešaka .
- Unapređenje VCL-a.
- Programiranje baza podataka sa višestrukim pristupom.

2.1.5. Glavne karakteristike Delphija 5

- ADO Dataset – alternativna tehnologija BDE-u, koristi Microsoft's Active Data Objects (ADO) tehnologiju.
- Data Module Designer – novi vizuelni alat koji olakšava lakše kreiranje Data Modula ("forme" za smeštanje komponenti za rad sa bazama).
- InterBase Express – integriše InterBase sa Delphijem i ne zahteva BDE (razvijanje baze podataka za rad sa udaljenim serverima)
- Unapređenja ActiveX kontrola.
- Forme se snimaju kao tekst fajlovi, tako da se mogu editovati iz bilo kog tekst editora.
- TeamSource – Alat koji omogućava da više ljudi radi na istom projektu.
- Novi interfejs za debugiranje aplikacija sa poboljšanim osobinama.
- i mnoge druge...

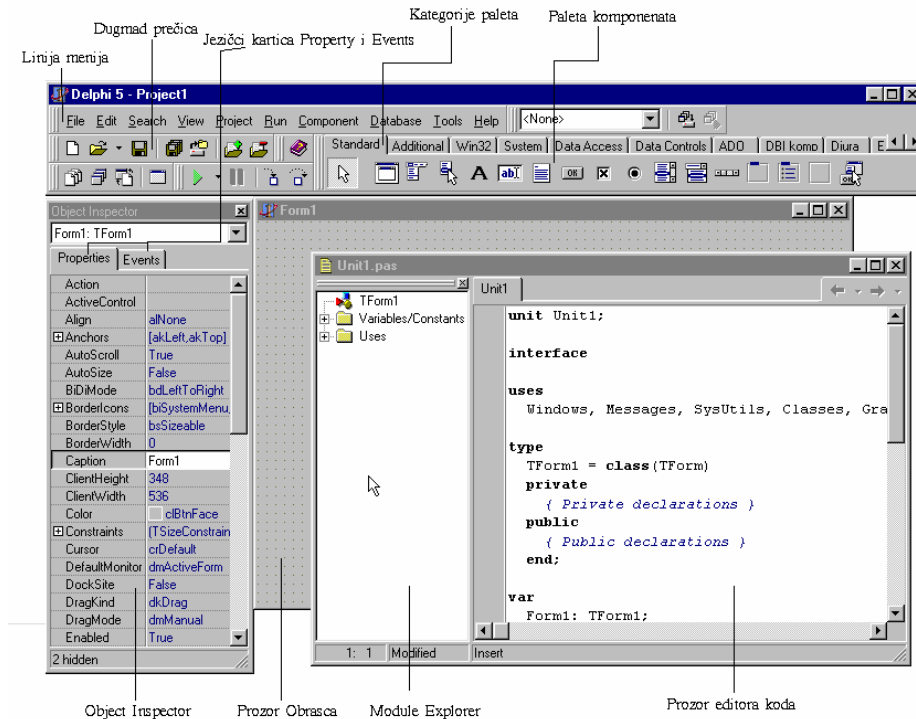
3. STRUKTURA DELPHIJA

3.1. Elementi Delphija

Kada se startuje Delphi, otvara se nekoliko prozora različite namene pomoću kojih se gradi aplikacija. To su:

Glavni prozor Delphija -Ovaj manji prozor je uvek smešten pri vrhu ekrana. Pomoću njega se podešavaju parametri programa, organizuju se alati za rad i vrši se kompajliranje (generisanje izvršnog koda). Svi ostali elementi sa kojima se radi su sinhronizovani pod kontrolom ovog glavnog prozora. Na njemu se nalaze sledeći elementi:

- Linija menija - *Ovde su smeštene opcije samog Delphija.*
- Dugmad prečica - *Tasteri za brz pristup najčešće korišćenim opcijama, bez potrebe da se traže u meniju. Služe da ubrza rad sa Delphijem.*
- Paleta komponenti - *Na njoj su prikazane ikone standardnih i specifičnih Windows komponenata koje se mogu smeštati po formi i tako graditi vizuelni izgled aplikacije. Paleta je podeljena na stranice u kojoj su komponente grupisane prema funkciji (Kategorije paleta).*



Slika 1. Delphi okruženje

Object Inspector – Prikaz svih svojstava odabrane komponente i omogućavanje menjanja istih. Kada se klikne mišem na bilo koji od objekata, u Object Inspectoru se pojavljuje lista njegovih svojstava koja se mogu menjati podešavajući tako objekat konkretnim potrebama.

- Kartice Properties i Events – *Prikaz svojstava komponenta ili prikaz događaja vezanih za komponente koje se nalaze na formi. Nalaze se na vrhu object inspektora, gde se nalazi u lista svih komponenti koje se nalaze na formi.*

Module Explorer – Prikaz klasa tekućeg modula, promenljivih, objekata, i drugih informacija.

Prozor editora koda – Mesto za unos i izmenu samog koda aplikacije koju trenutno radimo. Moguće je otvoriti više datoteka i raditi nezavisno sa svakom od njih (Module explorer – prikazuje svojstva za trenutno aktivnu datoteku). Pojedine linije su već na samom početku upisane u ovaj prozor. To je kod koji Delphi automatski unosi prilikom inicijalnog kreiranja kostura programa.

Kada se počne sa razvijanjem novog projekta, automatski se generiše prvi prozor buduće aplikacije. Ovaj objekat se u Delphiju naziva forma. Nova forma na početku nema na sebi nijednu komponentu, ali ima sve osobine standardnog Windows prozora: može se pomerati po ekranu, smanjivati i uvećavati, poseduje naslovnu liniju, okvir i radnu površinu (Canvas). Delphi automatski generiše sav potreban kod aplikacije da bi ona imala ovaj inicijalni prozor, tako da je moguće odmah startovati program - pri izvršavanju će se pojaviti ovaj prozor koji će biti prazan jer na njega još nije postavljena nijedna komponenta. Da bi program radio nešto korisno, potrebno je na njega smestiti komponente po potrebi i napisati

programski kod za određene događaje. Generisanje inicijalnog koda za događaje je automatsko.

Na formi se nalazi mreža tačaka koja se ne vidi prilikom izvršavanja programa, već samo dok se radi u Delphiju. Ova mreža se naziva Grid i ona služi kao pomoć pri postavljanju i manuelnom poravnavanju komponenti (mada postoji i automatsko poravnavanje komponenti), dajući vizuelni pregled radne površine.

3.2. Delphi kao alatka za vizuelno programiranje

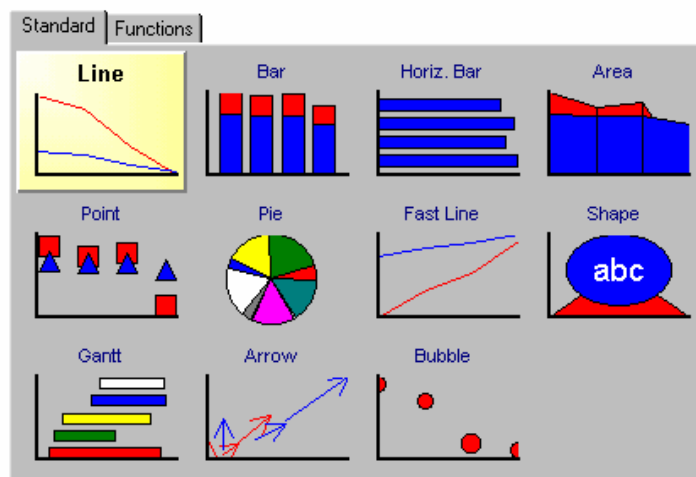
Okruženje za vizuelno programiranje, kakvo je Delphi omogućava da korišćenjem miša i seriju specijalizovanih vizuelnih i nevizuelnih komponenata, brzo i jednostavno projektujete aplikaciju. Imam iskustvo rada u Visual C++ koji omogućava sve isto kao i Delphi, ali način na koji to uradite u Delphiju je mnogo lakši a i utrošeno vreme za rešavanje istog problema je mnogo manje. To se posebno odnosi na rad sa bazama podataka, o čemu će kasnije biti reči.

Delphi ima nekoliko specijalizovanih alata koji ubrzavaju stvaranje aplikacija. Na primer kada radimo sa bazama podataka možemo koristiti **Database form wizard** za automatsko generisanje formi koje pokazuju jedan-prema-više relacije između tabela. Tako da možemo napraviti aplikaciju za rad sa bazama podataka bez pisanja koda, samo povezivanjem komponenti i odgovaranjem na pitanja koja wizard postavlja (naravno ovo se ne odnosi na ozbiljnije programe).

Query Builder - uključuje vizuelne alatke koje kreiraju kompleksne SQL naredbe.

QuickReport wizard – za prikaz i štampanje podataka iz baze podataka. Jednostavno se odgovara na postavljena pitanja i kroz minut vremena imate gotovu aplikaciju (bez upisa koda) koja omogućava preview/print podataka iz baze kao i snimanje u html, excel, txt i svom internom formatu. Ovaj prikaz ne mora da se odnosi na prikaz podataka iz jedne tabele već je moguće koristiti i naredbe SQL-a za izbor potrebnih podataka iz više tabela.

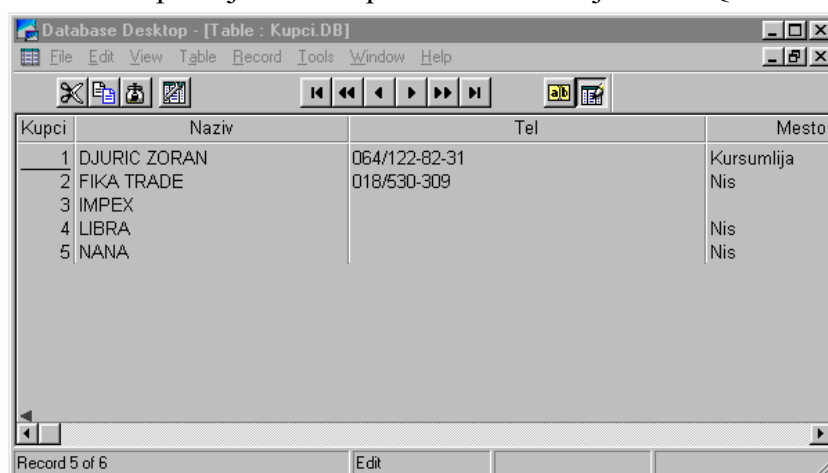
TeeChart wizard – za prikaz podataka numeričkih iz baze podataka ali u obliku grafikona (2D ili 3D) (otprilike 10-tak različitih oblika grafikona kao što se može videti sa *slike 2*. Takođe je moguće odštampati grafikon, ili ga snimiti u obliku datoteke na hard disk.



Slika 2. TeeChart wizard – mogući izgledi grafikona

Database Desktop – DBD

DataBase Desktop (DBD) je alatka za rad sa bazom podataka. Ona nam omogućava kreiranje upita, kreiranje, restrukturiranje, indeksiranje, modifikovanje i kopiranje baze podataka, tabela, uključujući datoteke Paradox ili dBase, kao i SQL tabele. Nije potrebno da posedujemo Paradox ili dBase da bismo koristili DBD sa desktop datotekama koje su u ovom formatu. DBD može kopirati podatke i podatke o rečniku podataka iz jednog formata u drugi. Npr. moguće je kopirati Paradox-ove tabele u postojeću bazu podataka na udaljenom SQL serveru.



Slika 3. Database Desktop sa otvorenom tabelom Kupci

3.3. Datoteke koje Delphi generiše

Tipična Delphi aplikacija se sastoji od velikog broja datoteka različitih tipova. Svako ime datoteke se završava nastavkom koji identifikuje sadržaj datoteke.

- **.č** datoteke – su rezervne kopije ili izmenjene i snimljene datoteke
- **.bpg** datoteke – *Borland Project Group* datoteke, koje mogu pamtit grupu projekata, u stvari su prikrivene tekstualne datoteke naredbi komandne linije

Make (sadrži liste zavisnosti modula i pruža neophodne komande i informacije za prevođenje i povezivanje višemodulske aplikacije).

- **.dcr** datoteke – *Delphi Component Resource* datoteke sadrže ikonu komponente u obliku u kojem se pojavljuje na VCL paleti.
- **.dcu** datoteke – Ove datoteke sadrže prevedeni kod i podatke za programske module.
- **.dfm** datoteke – Ove datoteke sadrže tekstualni opis formi.
- **.dof** datoteke – Ove datoteke sadrže opcije projekta (vezano za prevođenje, linkovanje,...)
- **.dpr** datoteke – Skraćenica za Delphi projekt; to je datoteka izvornog koda projekta.
- **.exe** datoteke – Datoteke izvršnog koda.
- **.pas** datoteke – Sadrže izvorni kod u Pascalu.
- **.res** datoteke – Sadrže binarne resurse koje program koristi (ikone, bitmapirane slike, tekst,...)

4. BAZE PODATAKA

4.1. Potreba za standardom pri radu sa bazama podataka

U ranim danima PC-a, programi koji su radili sa bazama podataka, pisani su da bi radili na jednom računar i podržavali su samo jednu bazu podataka. Pošto je tada PC bio na niskom stadijumu razvoja, (većina aplikacija je i dalje bila napisana za veće računare) baze podataka su retko, ako su uopšte i bile menjane. Ovo je zadovoljavalo potrebe dugi niz godina. Međutim kako je sve više i više kompanija prihvatilo PC kao platformu na kojoj će raditi njen softver, javilo se nekoliko problema koji nisu mogli biti efikasno rešeni bez postojanja standarda :

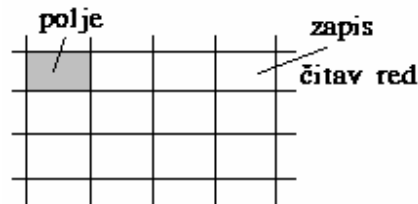
- Proširivost (engl. *scalability*) – Kako je PC postajao sve prisutniji na radnim stolovima, došlo je vreme kada je trebalo da aplikacije budu sposobne za rad u mreži. Drugim rečima, trebalo je da se aplikacije izvršavaju na jednom mestu i da imaju vezu sa bazom podataka koja radi na serveru. Ako je aplikacija bila napravljena za samostalni rad, to je najčešće značilo da mora biti prilagođena drugoj bazi (onoj koja je omogućavala više korisnika istovremeno). To je, opet, značilo dodatne troškove oko ponovnog pisanja aplikacija.
- Interoperabilnost (engl. *interoperability*) – Softverske kompanije su morale da se odluče za određenu bazu. Ako je kompanija želela više od jedne ciljane baze, trebalo je razvijati posebnu verziju softvera za svaku željenu bazu podataka. Očigledno ovo je bilo veoma skupo. Ono što je bilo potrebno jeste mogućnost da se softver napiše na takav način da može da podrži vezu ka bilo kojoj bazi podataka, bez potrebe za ponovnim pisanjem programa.
- Cena razvoja softvera – Pošto je API (engl. *application programming interface*) za bazu podataka skoro uvek bio specijalizovan (specifičan za bazu), ako je neka kompanija želela da razvije softver za korišćenje baze podataka A, bio joj je potreban ekspert za A. Dalje, ako je kompanija želela da razvija softver za bazu B, trebao joj je ekspert i za tu bazu podataka. Ovo je imalo za posledicu da je bilo skupo i teško pronaći programera. Bio je potreban standard API za baze podataka. Onda su kompanije mogle da traže programera koji je poznao standardni API koji ima mogućnost rada sa više baza podataka. Na taj način, bilo bi mnogo lakše i pronaći programera i sniziti cene održavanja programa.

4.2. Terminologija baza podataka

Evo nekih osnovnih definicija koje se koriste pri radu sa bazama podataka :

- **Polje** (engl. *field*) – Najmanji element podataka u bazi podataka. Polje ima *ime* i *tip*; na primer, tip polja nazvanog *Ime_i_prezime* biće tekstualnog tipa. Primeri tipova podataka koji se mogu koristiti u Delphiju su tekstualni, celobrojni, realni, novčani iznos, datum, vreme, grafički (za pamćenje slike u bazi), memo (tekstualni velicine 32 KB), logički, binarni, byte,...

- **Zapis** (engl.*record*) – Kolekcija polja koja sadrže srodne informacije; na primer zapis o radniku u nekoj firmi mogao bi da sadrži polja *ID_Radnika*, *Ime_radnika*, *Prezime_radnika*, *mat_br*, *plata*,...
- **Tabela** (engl.*table*) – Kolekcija zapisa koji svi imaju istu strukturu. Kada se tabela pokazuje u obliku rešetke, svaki red je jedan zapis, a kolona je jedno polje (*Slika 4*)



Slika 4. Tabela u obliku rešetke

- **Baza podataka** (engl. *database*) – Kolekcija povezanih tabela, obično određena direktorijumom koji sadrži te tabele (kod paradox tabela koje se najčešće koriste u radu sa BDE-om)
- **Indeks** (engl.*index*) – Indeks je polje koje se koristi za logičko sortiranje slogova tabele. Pri umetanju nekog sloga u sredinu datoteke, morala bi da se preraspodeli cela tabela što je duga operacija. Zbog toga se koriste indeksne datoteke koje čuvaju logički redosled slogova tabele tako da BDE isporučuje slogove u redosledu definisanom u indeksnoj datoteci. Svaka promena stanja tabele reflektuje se u indeksnoj datoteci koja je uvek ažurna. Indeksne informacije se najčešće nalaze u posebnoj datoteci sa ekstenzijom .PX (za Paradox tabele), odnosno .MDX (za dBase tabele). Za neke druge tipove baza podataka podaci o indeksima su smešteni zajedno sa tabelama ili kompletnom bazom podataka u istoj datoteci (na primer MS Access).
- **Primarni indeks** (engl. *primary index*) – Kontrolise redosled u kome se zapisi u bazi podataka prikazuju i zahtevaju ga mnoge funkcije za rad sa bazama podataka. Polje primarnog indeksa mora biti jedinstveno; to znači da ne mogu postojati dva zapisa koji imaju istu vrednost u tom polju. Primarni indeks može da sadrži i više polja, znači ne mogu postojati više zapisa sa istim vrednostima u tim poljima.
- **Sekundarni indeks** (engl.*secondary index*) – Tip indeksa koji može biti zasnovan na kombinaciji polja i ne mora biti jedinstven. Može se koristiti pri povezivanju tabela, pri prikazu podataka u rastućem/opadajućem redosledu,...
- **Upit** (engl. *query*) – Izraz u SQL-u koji bira zapise i polja iz jedne ili više tabela. Koristi se da bi se dobile određene informacije iz jedne ili više tabela; na primer spisak svih kupaca koji nisu platili račun, ili dugovanja kupca za period vremena.

Svaka baza podataka ima fizičku i logičku strukturu. Podaci su smešteni na disku u obliku datoteka. Ove datoteke mogu biti različite strukture, što zavisi od primenjenog formata zapisa koji je upotrebljen za njihovo kreiranje. Način na koji su podaci smešteni na disku naziva se fizička struktura podataka.

Ono što podaci stvarno predstavljaju nazivamo logičkom strukturom baze podataka. Za korisnike je bitna i jedino vidljiva logička struktura baze podataka.

Podaci koji se odnose na jednu zajedničku primenu smešteni su u jednu ili više datoteka na disku. Postoji veliki broj standarda u ovoj oblasti, koje primenjuju različiti proizvođači i programski paketi, tako da podaci mogu biti smešteni na različite načine. Vrsta fizičke organizacije podataka na disku je jako bitna za programera koji piše aplikaciju kako bi stvorio vezu između samih podataka i njihove logičke prezentacije koju vidi korisnik.

4.3. Komponente za rad sa bazama podataka

Delphi je programski paket koji je u potpunosti dovoljan za pravljenje svih programa koji rade pod Windowsom, ali je najbolji alat za pravljenje programa koji rade sa bazama podataka. Pomoću njega se može pisati softver praktično za sve vrste baza podataka : dBASE, Paradox i ODBC sisteme (npr. Microsoft Access). Takođe se mogu praviti i klijentsko-serverski programi za udaljene servere podataka, koji mogu da rade na svim platformama. Pored komponenti za rad sa lokalnim bazama podataka, postoje i komponente koje olakšavaju pristup udaljenim serverima sa bazama podataka kao što su : Oracle, Sysbase, Microsoft SQL server i Informix.

Paleta komponentata na kojima se nalaze komponente za pristup bazama podataka su *Data Access*, *ADO* i *InterBase*.

Na paleti *Data Access* se nalaze komponente koje najčešće koristimo pri radu sa lokalnim bazama podataka.

Na paleti *ADO* nalaze se komponente za pristup bazama podataka korišćenjem ADO tehnologije (bez korišćenja BDE-a).

Na paleti *InterBase* nalaze se komponente za pristup bazama podataka na udaljenim serverima.

Za prikazivanje i editovanje podataka iz baze koristimo komponente sa *Data Controls (DC)* stranice paleta komponenti. Većina *Data Controls* komponenti predstavlja verzije standardnih komponenti koje su u stanju da prikazuju podatke iz baza, ali se od standardnih komponenti razlikuju drastično. Ove kontrole nazivamo i data-aware (DA) ('svesne su šta pokazuju'). Ove DA kontrole izvode prikaz podataka iz izvorne baze podataka koja je izvan aplikacije, a mogu i da šalju i da primaju podatke iz izvora podataka. DC su DA za vreme kreiranja programa, što znači da kad povežemo komponentu na aktivni izvor podataka za vreme kreiranja programa, odmah smo u stanju da vidimo "žive" podatke u kontrolama. DC su povezane sa tabelom baze podataka preko osobine *DataSource*. Osobina *DataSource* specificira ime komponente *DataSource* iz koje kontrola dobija podatke (*DataSource* je povezan sa *Table*, *Query*, *ADOTable*, *ADOQuery*, ili nekom drugom komponentom koja pristupa bazi podataka).

Sledi spisak komponenti koje se u Delphi-ju najčešće koriste za rad sa bazama podataka (stranice *Data Access* i *Data Controls* u paleti komponentata) :

<i>BatchMove</i>	Izvodi paketne operacije nad tabelama (kopiranje skupa podataka, dodavanje podataka iz jednog skupa podataka u drugi, ažuriranje i brisanje,...). Najpre se Source i Destination svojstvima definišu dve tabele, od kojih prva predstavlja izvor odakle se uzimaju podaci, a druga mesto na koje će podaci biti smešteni. Pomoću Mode svojstva određuje se tip operacije. U slučaju da su izvorišna i odredišna tabela različitih struktura, svojstvom Mappings se određuju pravila dodeljivanja vrednosti poljima. Zadana operacija se zatim izvršava metodom Execute.
<i>Database</i>	Prijavljivanje na server (obrada lozinki, alijasa, parametri konekcije...)
<i>DataSource</i>	Povezuje nevizuelne komponente koje ostvaruju vezu sa tabelama sa vizuelnim koje prikazuju sadržaj tabela ili SQL upita.
<i>DBChart</i>	Pravljenje grafikona od podataka iz tabele
<i>DBCheckBox, DBComboBox, DBEdit, DBImage, DBListBox, DBMemo, DBRadioGroup, DBRichEdit</i>	Ovo su komponente prikazuju podatke različitog tipa iz baze podataka (tekst, brojeve, slike, true/false,...)
<i>DBCtrlGrid</i>	Skup redova koji se mogu pomerati i od kojih svaki predstavlja jedan zapis iz baze podataka. Svaki red može imati jedan ili više objekata koji predstavljaju polja zapisa iz baze.
<i>DBGrid</i>	Mreža redova i kolona koja u svakom redu prikazuje po jedan zapis iz baze podataka. Obično se koristi zajedno sa objektom DBNavigator.
<i>DBNavigator</i>	Database navigator vizuelno izgleda kao skup tastera specifične namene. Svaki taster korespondira sa metodom istovetnog naziva table ili query komponente sa kojom je povezan data source komponentom. Korisnik time dobija kontrolu nad operacijama upisa, brisanja izmene i kretanja po slogovima bez potrebe da se pišu posebne procedure. Database navigator se najčešće koristi uz druge database komponente povezane sa istom data source komponentom i u kojima se prikazuju i unose podaci u polja. Upravljanje podacima se zatim obavlja pomoću database navigatora. Pojedini tasteri mogu se proglasiti nevidljivima, kako korisnik ne bi mogao da ih koristi. Tasteri database navigator komponente: <i>First</i> Pomera tabelu na prvi slog. <i>Prior</i> Pomera tabelu za jedan slog unazad. <i>Next</i> Pomera tabelu za jedan slog unapred. <i>Last</i> Pomera tabelu na poslednji slog.

	<i>Insert</i> Umeće slog u tabelu. <i>Delete</i> Briše slog iz tabele. <i>Edit</i> Omogućava promenu polja tekućeg sloga. <i>Post</i> Snima sadržaj sloga u tabelu. <i>Cancel</i> Poništava izmene na slogu. <i>Refresh</i> Osvežava podatke na database komponentama.
<i>Query</i>	šalje SQL komande sistemu BDE ili SQL serveru.
<i>Session</i>	Ova komponenta se automatski kreira na početku izvršavanja svake Delphi aplikacije. Iako je nevidljiva i retko iskorišćena u programu, ona u pozadini kontroliše rad sa bazama podataka na globalnom nivou.
<i>StoredProc</i>	Omogućava programima da izvršavaju procedure smeštene na serverima na kojima se nalaze baze podataka (vezano za klijentsko serverske programe).
<i>Table</i>	Pristup tabeli preko BDE.
<i>UpdateSQL</i>	Za ažuriranje skupova podataka dobijenih preko SQL servera.

4.3.1. **Table i Query**

Dve najznačajnije komponente za pristup podacima (preko BDE-a) su *Table* i *Query*. Sledi malo širi opis svojstava koje ove komponente imaju:

<i>DatabaseName</i>	Određuje bazu podataka koja sadrži ovu tabelu; to je ili alijas ili ime direktorijuma koji sadrži traženu tabelu (alias – kasnije objašnjeno značenje u tekstu)
<i>TableName</i>	Ime tabele (ime datoteke na disku koja predstavlja određenu tabelu) (samo kod table)
<i>Active</i>	Svojstvo koje ima vrednosti <i>true</i> (tabela otvorena) i <i>false</i> (tabela zatvorena). Moguće je otvarati i zatvarati tabelu i tokom izrade aplikacije. Ako tabela ima primarni ključ, podaci će se pojaviti u redosledu formiranom pomoću tog ključa. Ako umesto tog svojstva želimo da upotrebimo sekundarni, koristimo sledeće svojstvo,
<i>IndexName</i>	Omogućava korišćenje sekundarnih indeksa i prikaz podataka u zavisnosti od sekundarnog indeksa (samo kod Table).
<i>IndexDefs</i>	Objekat tipa <i>TIndexDefs</i> u kome se nalaze sve informacije o svim indeksima tabele. Ovo svojstvo se može samo čitati (samo kod table).
<i>IndexFieldCount</i>	Broj polja koja grade aktivan indeks (samo kod table).
<i>UpdateMode</i>	Određuje uslove za promenu sloga u višekorisničkom okruženju kako bi se izbeglo istovremeno ažuriranje više korisnika (samo kod table).
<i>CashedUpdates</i>	Keširane promene, sprečavaju program da stalno

	uznemirava mrežu sa malo podataka. Program sa keširanim promenama skladišti promene lokalno, a onda ih ođednom pusti kroz mrežu. Program treba da vodi računa o tome da i neki drugi program može menjati podatke u istoj bazi, pa može doći do nedozvoljenih promena u bazi, tako da kad god se radi o bazi koju koristi više programa istovremeno, treba postaviti vrednost ovog svojstva na false.
<i>AutoCalcFields</i>	Svaki put kada se promeni tekući slog, poziva se metoda <i>OnCalcFields</i> u kojoj se mogu dodeliti vrednosti sračunatim poljima. Ovo svojstvo određuje da li se ovaj događaj poziva automatski i tokom rada sa slogom, svaki put kada se promeni vrednost bilo kog polja. Na ovaj način omogućava se održavanje ažurnog stanja sračunatih polja.
<i>Exclusive</i>	Određuje da li drugi korisnici u višekorisničkom okruženju mogu pristupati tabeli. <i>true</i> označava zabranu pristupa tabeli za druge korisnike.
<i>Filter</i>	Filtriranje podataka tako da prikazuje samo podskup podataka iz tabele; na primer stavimo : <i>Filter:= 'Plata=5000'</i> u nekoj tabeli radnici i imaćemo prikaz svih radnika sa platom od 5000.
<i>Filtered</i>	Vrednost <i>false</i> znači da zanemarujemo filter, tj. vidimo sve podatke, a <i>true</i> da vidimo podskup podataka ođeđenog filter-om.
<i>MasterSource</i>	Koristimo kada hoćemo da povežemo više tabela po nekom ključu (indeksu); na primer imamo tabelu <i>Kupci</i> sa podacima o kupcima i <i>Narudzbine</i> sa podacima koji je kupac šta narućivao. Ova komponenta nam omogućava da kretanjem kroz tabelu <i>Kupci</i> se automatski vrši i kretanje kroz tabelu <i>Narudzbine</i> (automatski se pronalaze narudzbine za trenutnog kupca – bez pisanja koda) – kasnije detaljno objašnjeno
<i>MasterFields</i>	Odnosi se na prethodno svojstvo i kaže nam po kojoj ćemo vrednosti polja vršiti povezivanje tabela; na primer za prethodni slučaj to može biti polje <i>ImeKupca</i> koje postoji u obe tabele.
<i>ReadOnly</i>	Vrednost <i>true</i> , znači možemo samo čitati iz tabele, a <i>false</i> da možemo čitati i upisivati u tabelu.
<i>SQL</i>	Ovo svojstvo sadrži tekstualni upit za prikaz ođeđenog skupa podataka iz jedne ili više tabela. (samo kod Query)

Kada se table komponenta postavi na formu i podese joj se navedene vrednosti u Object Inspector-u, može se svojstvom *Active* odrediti da li je tabela otvorena ili ne. Ako se ovo svojstvo postavi na *true*, Delphi pokušava da otvori tabelu. Ako se tabela uspešno otvori na formi se čak i prilikom dizajna vide podaci iz baze podataka u database komponentama. U slučaju neuspeha, dobija se

obaveštenje o tome što znači da tabela ne postoji, ili je neko od svojstava postavljeno na pogrešnu vrednost.

Tabela se alternativno može otvoriti pozivanjem metoda *Open* ili *Close*:

```
Procedure Form1. FormCreate (Sender: TObject);  
begin  
    Table1. Open  
end
```

Na ovaj način obezbeđuje se automatsko otvaranje tabele čim se kreira sama forma. Eksplicitno zatvaranje tabele na kraju izvršavanja aplikacije nije potrebno, jer se ta radnja izvršava automatski.

4.3.1.1. Kretanje kroz tabelu

Kada je komponenta tabele kreirana i tabela otvorena, samo jedan slog je trenutno aktivan. U početku je to prvi slog tabele. Vrednosti polja do kojih se dolazi preko field komponentata uvek ukazuju na podatke tekućeg sloga. Prebacivanje na sledeći slog obavlja se pomoću procedure *Next*, a na prethodni procedurom *Prior*. Skok na prvi slog tabele se vrši procedurom *First*, a zadnji procedurom *Last*. Svojstva BOF i EOF se postavljaju na *true* kada je tekući slog prvi, odnosno zadnji u tabeli.

Primer: deo programa koji prolazi kroz celu tabelu Kupac :

```
while not tKupac.EOF do  
begin    //uraditi nešto sa tekućim slogom  
    tKupac.Next  
end;
```

Procedura *MoveBy* vrši pomeranje za zadati broj slogova unapred ili unazad

Primer:

```
tKupac.MoveBy (-3)
```

postavlja slog koji se nalazi 3 mesta pre tekućeg za novi tekući slog.

4.3.1.2. Određivanje sadržaja polja

Polja su u tabeli predstavljena nizom *TField* komponenti. Ovaj niz je smešten u *Field* svojstvu tako da se do vrednosti određenog polja može doći navođenjem *tKupac.Fields(n).Value*, pri čemu je *n* redni broj polja u tabeli. Umesto rednog broja za određivanje polja može se koristiti naziv polja upotrebom metode *FieldByName*:

```
tKupac.FieldByName('NazivPolja').Value:=vrednost
```

4.3.1.3. Pretraživanje tabele

Osim metoda koje omogućavaju pomeranje unapred i unazad za određeni broj slogova u tabeli, postoje metode koje prolaze kroz slogove tabele tražeći prvi slog koji ispunjava neki uslov. Funkcija *GotoKey* traži prvi slog koji ispunjava zadati uslov i postavlja ga za tekući:

```
With tKupac do  
begin  
    SetKey;  
    FieldByName ('ImeArtikla').AsString:='Vinjak rubin';
```

```

        If GotoKey then // uraditi nešto (slog sa traženim vrednostima je
nađen)
    end

```

Tabela se postavlja u *SetKey* stanje i određuju se vrednosti po kojima ćemo da tražimo u tabeli. Rezultat izvršavanja *GotoKey* metode je postavljanje tekućeg sloga na prvi slog u tabeli koji ispunjava uslov. *GotoKey* metod se ponaša kao funkcija - ona vraća *true* ako je slog nađen. U protivnom je *false* i tekući slog je i dalje onaj koji je to bio pre poziva metode.

GotoNearest se koristi kao i *GotoKey*, osim što se za tekući slog proglašava prvi slog u kome je vrednost odgovarajućeg polja veća ili jednaka traženoj vrednosti.

4.3.1.4. Promena vrednosti polja

Promena tekućeg sloga tabele se vrši tako što se tabela postavi u *Edit* stanje pozivom procedure *Edit*, izvrši dodela vrednosti slogovima i pozove procedura *Post* kojom se izmene čine konačnim i upisuje promenjeni slog u tabelu:

```

Edit;
FieldName('ImeRobe').AsString:='Vinjak rubin 5';
FieldName('Jm').AsString:='fla';
FieldName('Kolicina').AsFloat:=0;
FieldName('NabCena').AsFloat:=210;
FieldName('ProdCena').AsFloat:=300;
FieldName('Dobavljac').AsString:='Rubin_Krusevac';
Post;

```

Ako se umesto metode *Edit* upotrebi metoda *Insert*, neće biti promenjen tekući slog već će se umetnuti novi slog sa zadatim vrednostima ispred njega. Posle izvršenja metode *Post*, tabela koja je bila u *Edit* ili *Insert* stanju vraća se u osnovno *Browse* stanje.

Ako se umesto metode *Edit* upotrebi metoda *Append*, neće biti promenjen tekući slog već će se umetnuti novi slog sa zadatim vrednostima na kraju tabele. Posle izvršenja metode *Post*, tabela koja je bila u *Edit* ili *Insert* stanju vraća se u osnovno *Browse* stanje.

Metoda *Cancel*, ako se izvrši pre konačne potvrde metodom *Post*, ukida sve promene tekućeg sloga i postavlja tabelu u *Browse* stanje.

Primer dodatne provere pri upisu od strane programa:

```

Procedure Form1. BtnSnimiClick (Sender:TObject);
begin
    // deo koda gde se poljima dodeljuju vrednosti
    If MessageDlg ('Da li želite da upišete
podatke',mtQuestion,šmbOk,mbNo],0)=mrOk
    then Table1.Post
    else Table1.Cancel;
end;

```

Ako želimo da se nova vrednost sloga upiše, treba da pritisnemo taster OK.

Tabela se u svakom trenutku nalazi u jednom od stanja definisanim svojstvom *State*. Najčešća su :

<i>dsInactive</i>	Označava da tabela nije otvorena.
<i>dsBrowse</i>	Tabela je otvorena i vrednosti se mogu čitati.
<i>dsEdit</i>	Može doći do promene vrednosti polja upotrebom procedure Post.
<i>dsInsert</i>	Upotrebom procedure Post uključuje se novi slog koji se umeće pre tekućeg
<i>dsSetKey</i>	Promene vrednosti polja se ne odnose na slog već se zadaju vrednosti ključevima.
<i>dsFilter</i>	Svojstvo tabele <i>filtered</i> je postavljeno na true.
<i>dsOpening</i>	Tabela je u procesu otvaranja.
<i>dsCalcFields</i>	U toku je poziv događaja OnCalcFields.

Postoje još par stanja koja su trenutna. Prebacivanje tabele iz jednog u drugo stanje se ne može vršiti promenom svojstva *State*, već procedurama *Edit*, *Insert* i *SetKey*.

Tabela se u stanja *Inactive*, *Browse* i *CalcFields* prebacuje automatski i ne postoje posebne procedure za prebacivanja u ova stanja.

4.3.1.5. Master - detail relacija

To je odnos u kome se sadržaj određenog polja iz jedne tabele (engl. *master table*) koristi kao ključ za izbor sloga iz druge tabele (engl. *detail table*).

Tokom izvršavanja aplikacije može se definisati da je neka tabela detail, odnosno da njen tekući slog zavisi od sadržaja nekog polja njene master tabele. Master tabela se određuje svojstvom *MasterSource*, a ključno polje (ili više njih) po kome će se vršiti pozicioniranje *MasterFields*. *MasterSource* sadrži referencu na datasource komponentu koja ukazuje na master tabelu, dok se u *MasterFields* definišu nazivi polja odvojenih znakom (;).

```
DataSource1.TableName:= 'ulaz ';
...
Ulaz1.MasterSource:=DataSource1;
Ulaz1.MasterFields:= 'šifra_naloga';
```

Tabela ulaz sadrži podatke o nalogima (šifra naloga, datum, dobavljač,...) – za svaki nalog po jedan sloga tabela ulaz1 sadrži stavke naloga (šifra naloga, ime artikla, količina, nabavna cena,...) – za svaki nalog jedna ili više stavki. Ovim kodom smo povezali ove dve tabele, tako da kretanjem kroz tabelu ulaz automatski se krećemo i kroz tabelu ulaz1 (ne moramo da pišemo procedure za pretraživanje obe tabele već samo jedne, i kad nađemo traženi nalog, automatski smo pronašli i stavke vezane za taj nalog).

Master - detail odnos se može definisati i tokom dizajniranja, samo postavimo date vrednosti u određena svojstva.

4.3.2. Field komponenta

Ova komponenta ne postoji na paleti komponenti, ali je njena uloga izuzetno važna za rad sa bazama podataka u Delphiju. Svaka field komponenta predstavlja jedno polje u tekućem slogu tabele. Prilikom kreiranja table ili query komponente, automatski se kreira i niz field komponenti koji oslikava polja sloga tabele, i u programu im se pristupa preko svojstva *Fields* ili metode *FieldByName* table komponente.

Postoji nekoliko tipova objekata nasleđenih od osnovnog tipa *TField* (koji se sam za sebe ne koristi, već služi samo kao roditelj izvedenim tipovima), koji se koriste za razne formate podataka koji se mogu smestiti u polje: *TBooleanField* za smeštanje logičkih vrednosti (*true/false*), *TIntegerField* za smeštanje celih brojeva, *TDateTime* za smeštanje datuma i vremena, *TFloat* za realne vrednosti, itd. Naziv tipa je analogan tipu podatka koji se čuva u objektu.

Svi Field objekti imaju veći broj svojstava, procedura, funkcija i događaja koji se mogu iskoristiti u kodu za pristup i manipulaciju podacima pojedinih polja tekućeg sloga.

Sadržaj polja nalazi se u *Value* svojstvu, čiji tip zavisi od tipa podatka. Svojstva *AsBoolean*, *AsDateTime*, *AsFloat*, *AsInteger*, *AsString* mogu da vrše konverziju iz jednog tipa u drugi.

Primer: Neka u tabeli Roba postoji kolona *ImeRobe* i *ProdCena*.

```
Edit1.Text:=tRoba.FieldByName ('ImeRobe').AsString  
Edit2.Text:=tRoba.FieldByName ('ProdCena').AsString
```

tRoba je u ovom slučaju objekat tipa *TTable* koji predstavlja tabelu Roba. Svojstvo *FieldByName* vraća field zadatog naziva. Svojstvo *AsString* pretvara prodajnu cenu (tipa real) u čitljivi string, koji se može ispisati u edit boks Edit2.

Sledi spisak svojstava koji poseduje svaki field objekat sa objašnjenjem njihovih namena:

<i>Aligment</i>	Određuje da li se sadržaj polja u database komponenti prikazuje poravnat u levo, udesno ili centrirano.
<i>Calculated</i>	Određuje da li je polje sračunato. U slučaju da je ovo polje postavljeno na <i>false</i> , polje nije sračunato. Možemo programski da pravimo sračunata polja; na primer napravimo polje iznos koje je jednako polju <i>količina*Cena</i> i koristimo ga kao da to polje postoji u bazi (automatski se izražunava za svako polje u bazi). <i>Procedure Form1.Table1CalcFields (DataSet: TDataSet);</i> <i>begin</i> <i>With Table1 do</i> <i>Iznos.Value:=Količina.Value * Cena.Value</i>

	<i>end;</i>
<i>Currency</i>	Određuje da li se sadržaj polja prikazuje kao novčana vrednost.
<i>DataSet</i>	Table ili query komponenta kojoj field pristupa.
<i>DataSetSize</i>	Veličina podatka u bajtovima.
<i>DataSetType</i>	Tip podataka.
<i>Title</i>	Natpis koji se ispisuje kao naziv kolone kod korišćenja Database grid-a.
<i>DisplayText</i>	String koji se ispisuje u database komponenti.
<i>DisplayWidth</i>	Širina polja (u broju karaktera) koji se koristi da odredi širinu ćelije u slučaju korišćenja grid komponenti.
<i>EditMask</i>	Maska koja se koristi za unos sadržaja preko database komponenti. Možemo napraviti masku za unos datuma, celog broja, samo slova, samo broj sa određenim brojem cifara, novčano polje itd..
<i>FieldName</i>	Naziv polja u tabeli.
<i>Index</i>	Redni broj polja u tabeli, odnosno u nizu field objekata.
<i>CurValue</i>	Trenutna vrednost polja
<i>ReadOnly</i>	Određuje da li se sadržaj polja može samo čitati.

4.3.3. Database grid

Iz standardne *draw grid* komponente koja se koristi za tabelarni prikaz podataka izvedena je ova komponenta. *Database grid* poseduje dodatne mehanizme koji joj omogućavaju da automatski prikazuje podatke iz baze podataka kada se poveže sa nekom data source komponentom. Ova komponenta je najsveobuhvatniji i najlakši način da se omogući korisniku pregled i promena podataka iz baze.

Upotreba database grid komponente za vizuelno predstavljanje podataka je izuzetno pogodna. Svaka kolona ima jedno fiksno polje u kome se nalazi naziv polja, dok redovi predstavljaju slogove tabele. Aktivni slog je obeležen oznakom u prvoj koloni koja je takođe fiksna. Ako podaci nisu označeni samo za čitanje, oni se mogu menjati direktnim unosom novih vrednosti u ćelije database grid-a. Takođe, podešavanjem svojstava pojedinačnih TField komponenti (koje korespondiraju sa prikazanim kolonama), mogu se definisati maske za unos i formati prikaza podataka. Promenom tekućeg reda grid-a automatski se promenjeni slog ažurira u bazi podataka. Korišćenjem tastera CTRL+Del može se obrisati tekući slog.

4.4. Pravljenje programa za rad sa bazama podataka

Delphi-jeve komponente za rad sa bazama podataka standardizuju pristup bazama podataka raznih formata. To znači da programi mogu pristupati podacima iz dBASE-ovih datoteka, Paradoxovih tabela, tabela programa Microsoft Access i

drugih sistema za otvoreno povezivanje baza podataka (engl. Open Database Connectivity, ODBC) ili udaljenih SQL servera.

Znači bez obzira kojoj bazi podataka prisupate možete koristiti sve komponente koje Delphi nudi i u najvećem broju slučajeva ne morate menjati kod ili se izmene koda svode na promenu par redova.

4.5. Lokalne i client / server aplikacije

Baza podataka sa kojom radi Delphi aplikacija ne mora da se fizički nalazi na istom mestu gde i program. Računari su često organizovani u mrežu, pri čemu jedan računar (ili više njih) igra ulogu servera. Na njemu je smeštena baza podataka kao i specijalna server aplikacija koja obrađuje podatke prema zahtevima pojedinih udaljenih stanica - klijenata, računara povezanih sa serverom preko mrežnih kablova i specijalnih protokola za razmenu podataka. Na taj način omogućeno je da jednoj istoj bazi pristupa veliki broj korisnika.

Ovakva organizacija pristupa naziva se klijent - server (client - server) organizacija. Aplikacije koje pristupaju bazi podataka koja se nalazi na serveru moraju da imaju specijalne rutine i metode za obradu podataka. Podaci se ne prenose na stanicu, već se serveru preko mreže šalju komande u specifičnoj formi koji nalažu serverskom programu koji se izvršava na serveru da obradi podatke prema zahtevu.

Za pisanje client - server orjentisanih aplikacija potrebno je posedovati specijalnu verziju Delphija koja poseduje dodatne alate i komponente koje omogućuju kreiranje ovih aplikacija.

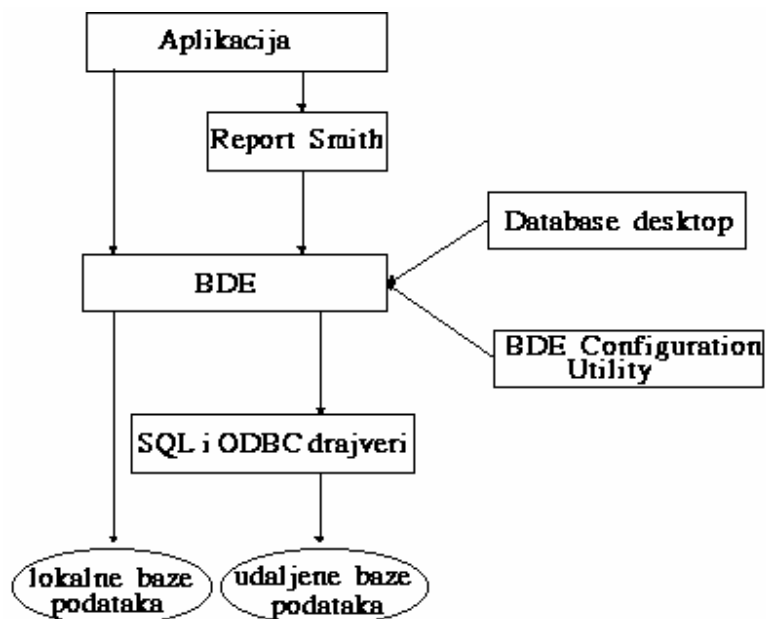
Za razliku od client - server, postoje i lokalne aplikacije kod kojih je baza podataka smeštena na lokalnom disku računara na kome se aplikacija za obradu podataka izvršava. Da bi se stvorila veza između datoteke na disku i komponente za pristup podacima u Delphiju potrebno je samo u odgovarajućem svojstvu navesti njen naziv na disku.

Uz standardnu verziju Delphija se isporučuje i LocalInterbase Server koji nam omogućava da pišemo program kao da radimo na udaljenom serveru. Kada završimo sa pisanjem i testiranjem programa, možemo bez imena u kodu postaviti da program radi sa pravim Interbase serverom (samo promenom alijasa).

4.6. Borland database engine (BDE)

BDE - Borlandov mehanizam za baze podataka je skup biblioteka koji tokom rada aplikacije nevidljivo u pozadini prevodi, snima i učitava podatke iz određenog formata u upotrebljivi oblik i obrnuto.

To je u stvari skup funkcija niskog nivoa (slično Windows API-ju) koje se direktno obraćaju podacima i serviraju ih aplikacijama, tačnije komponente kao što su table ili query. Ni programer ni korisnik ne moraju biti svesni da se rad sa podacima obavlja preko ovog posrednika.



Slika 5. Okruženje pri radu sa bazama podataka u Delphiju

Međutim da bi se veza sa bazom podataka ostvarila, BDE mora biti instaliran na računaru koji koristi ove komponente. Prilikom instalacije Delphija automatski se instalira i BDE tako da je pravljenje baza odmah moguće. Kada je program završen, nije dovoljna samo .EXE datoteka da bi se program izvršavao na svakom računaru već se mora instalirati i BDE (tj negde oko 10 MB za potpunu instalaciju BDE-a).

Pomoćni program *BDE Configuration Utility* je deo BDE i samim tim i Delphija. Njime možemo podešavati razne parametre rada BDE biblioteke, kao što je npr. spisak podržanih tipova baza podataka i spisak definisanih alijasa,...

ALIAS je proizvoljan naziv koji dodeljujemo bilo kom direktorijumu na disku. Prilikom kreiranja i korišćenja programa, umesto da za poziciju baze podataka navodimo direktorijum, dovoljno je navesti njegov alias definisan u BDE konfiguracionom fajlu. Ako se pozicija podataka promeni, dovoljno je promeniti definiciju alijasa da bi mesto gde se nalazi BP bilo locirano bez potrebe da se menja i ponovo kompajlira sam program.

BDE je dosta brz. Evo nekih poređenja koje sam uspeo da nabavim (*tabela 1 i tabela 2*).

Za poređenje korišćenja je tabela koja sadrži polja : Ime, prezime, telefonski broj i adresu.

Telefon je *integer* (celobrojnog) tipa a sva ostala polja su *character* (tekstualnog) tipa.

Broj slogova	Delphi	VB	Delphi	VB	Delphi	VB	Delphi	VB
	Upis	Upis	čitanje	čitanje	Sortiranje	Sortiranje	Ukupno	Ukupno
100	2	2	0	1	0	0	2	3
1000	16	70	6	23	1	22	23	115
2000	33	141	12	46	4	84	49	271
3000	50	227	17	69	8	189	75	485
4000	67	297	23	77	15	294	105	668

Tabela 1. Poređenje brzine Visual Basica i Delphija pri radu sa bazama podataka

Operacija	Delphi	PowerBuilder
Upotreba Query-ja za čitanje 20000 slogova iz tabele. podrazumeva se i traženje i filtriranje slogova.	4.6	70.9
Upis sloga	1.4	1.3
Filtriranje 20000 slogova	3.0	6.2
Traženje vrednosti u tabeli sa 20000 slogova	1.1	1.5
Ažuriranje sloga	1.5	1.1

Tabela 2. Poređenje Delphija i PowerBuildera pri radu sa bazama podataka (vrednosti su u sekundama)

5. OPIS PROBLEMA

5.1. ZADATAK

U praktičnom delu rada projektovana je Windows aplikacija razvijena u Delphi okruženju, koja praktično realizuje prethodno opisani problem rada sa bazama podataka. Zadatak je bio realizovati datu aplikaciju koja radi sa bazama podataka na lokalnom disku i koja će obuhvatiti zahteve kao što su unos podataka preko maske za unos, ažuriranje baza podataka, traženje kroz bazu, pregled i štampanje izveštaja. Jedinstveni direktorijum gde se tabele nalaze, definisan je preko aliasa 'diplomski' u programu *BDE Configuration Utility*.

Korisnik ne mora da poznaje strukture tabela, mada ima mogućnost njihovog pregleda.

Aplikacija se koristi se za vođenje evidencije artikala u nekoj firmi. To bi u najkraćim crtama bio opis aplikacije.

5.2. OBJAŠNJENJE STRUKTURE PROGRAMA

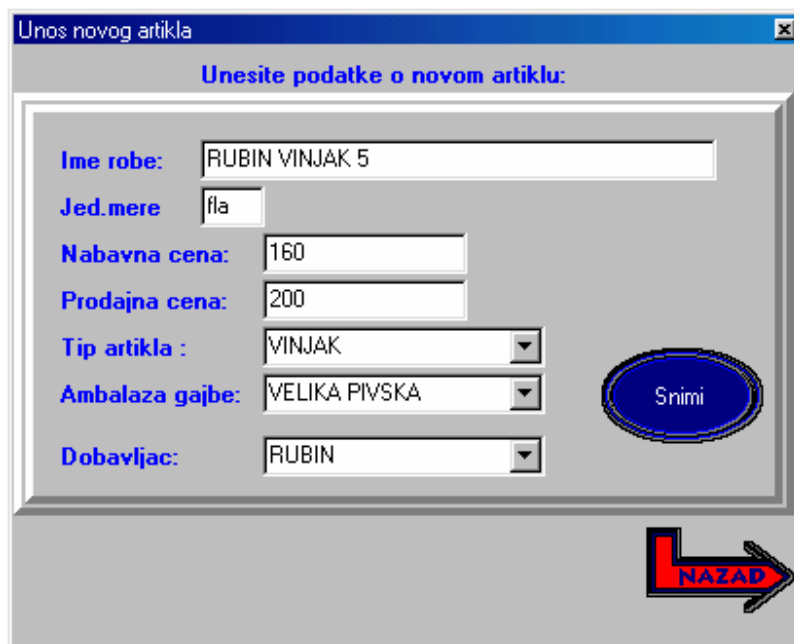
Program razvijen u ovom radu, se sastoji od 22 forme, jednog *Data Modula* i jednog *About Boxa* praćenih njihovim Pascalskim kodom, kao i kodom koji se odnosi na glavni program. Zadatak programa je da prati poslovanje neke firme i to :

- Stanje robe na zalihama (ime arikla, količina, nabavna cena, prodajna cena, naziv dobavljača, ambalaža (pivska, sok, kisela voda,...), tip artikla (npr. slatkiši, vina, rakije,...),
- Zaduživanje ambalaže pri prodaji ili kupovanju arikala koja imaju ambalažu (da znamo koliko firma duguje dobavljačima i koliko joj duguje svaki kupac, svake ambalaže ponaosob),
- Finansijsko zaduživanje (dugovanje firme dobavljačima, i koliko nam duguju kupci svaki ponaosob),
- Pregled ulaza arikala (po imenu, po dobavljačima, za period vremena, po tipu artikla,...),
- štampanje svih pregleda,
- Snimanje svih pregleda u tekstualnu datoteku na disku,
- Pregled svih ulaza/izlaza arikala iz firme,

Zbog obimnosti koda, u dogovoru sa profesorom prikazaću deo formi i koda.

5.2.1. Forma - Unos artikla

Izgled forme je kao na slici 6:



Slika 6. Forma za unos novog artikla

U ovoj formi se vrši unos novog artikla. Unesemo potrebne podatke i pritisnemo dugme *Snimi*. Ako ne želimo da snimimo, pritisnemo dugme *Nazad*.

Kod za snimanje ovih podataka u tabelu izgleda :

```
procedure TfUnosRobe.BtnSnimiClick(Sender: TObject);
var kraj : boolean;
    i : integer;
begin
    with baze.tRoba do
        if ((Length(EditImeRobe.Text)>0) and
            (Length(ComboBoxDobavljac.Text)>0)) then
            begin
                SetKey; //preraga tabele da li slučajno već postoji artikal sa tim imenom
                FieldByName('ImeRobe').AsString:=Trim(EditImeRobe.Text);

                FieldByName('NabCena').AsFloat:=StrToFloat(Trim(EditNabCena.Text));
                if GotoKey then
                    begin
                        MessageDlg('Vec postoji takav artikl!', mtInformation, šmbOk], 0);
                        Exit;
                    end;
                // snimamo u bazu
                Append; //dodavanje novog sloga (praznog pa ga napunimo sa
                podacima)
                FieldByName('ImeRobe').AsString:=Trim(EditImeRobe.Text);
                FieldByName('Jm').AsString:=Trim(EditJm.Text);
```

```

FieldByName('Kolicina').AsFloat:=0;
FieldByName('NabCena').AsFloat:=StrToFloat(EditNabCena.Text);
FieldByName('ProdCena').AsFloat:=StrToFloat(EditProdCena.Text);
FieldByName('Dobavljac').AsString:=Trim(ComboBoxDobavljac.Text);
FieldByName('Ambalaza').AsString:=Trim(ComboBoxAmbalaza.Text);

FieldByName('ImeTipa').AsString:=Trim(UpperCase(ComboBoxTip.Text));
Post; // upis u bazu
ObrisiEde; //brisanje svih podataka iz vizuelnih komponenti koje smo
uneli
end
else      MessageDlg('Unesite      ime      robe      i      ime
dobavljacka',mtInformation,šmbOk],0);
end;

```

1. Funkcija *StrToFloat* – vrši pretvaranje tipa *string* u tip *real*.
2. Funkcija *Trim* – eliminiše blanko znake na početku i na kraju stringa
3. Funkcija *UpperCase* – pretvara sva slova u velika.

Tabela Roba ima sledeća polja :

Ime polja	Tip	Opis polja
<i>ImeRobe</i>	Character 30	Naziv artikla
<i>Jm</i>	Character 3	Jedinica mere (fla, kom, lit,...)
<i>NabCena</i>	Number	Nabavna cena artikla
<i>ProdCena</i>	Number	Prodajna cena artikla
<i>Količina</i>	Number	Stanje artikla na lageru
<i>Dobavljač</i>	Character 30	Ime dobavljača datog artikla
<i>ImeTipa</i>	Character 15	Tip artikla : Slatkiš, pivo, rakija,...
<i>Ambalaža</i>	Character 15	Vrsta ambalaže : Kisela voda, pivo, sok,...

Primarni ključ je *ImeRobe* i on je jedinstven u tabeli. Na osnovu njega vršimo pretraživanje po tabeli.

5.2.2. Forma za unos ulaznog naloga

U sledećoj formi se vrši unos naloga. Na primer dođe nam nekoliko različitih artikala od nekog dobavljača. U ovoj formi unesemo pod kojom šifrom je nalog došao, datum prispeća, ime dobavljača, kao i artike koji su stigli (ime artikla, količina, cena, ambalaža) i sve to snimimo u bazu. Unos arikala je iz postojeće baze *Roba*. Jednostavnim klikom miša ili kursorskim strelicama izaberemo arikal i unesemo količinu. Posle unosa podataka pritisnemo dugme *Ubaci* (ili taster F5), pa unesemo novu stavku,... Ako artikal ne postoji pritisnemo dugme *Novi arikal* i unesemo novi artikal u bazi korišćenjem prethodne forme, pa nastavimo sa unošenjem stavki. Za snimanje koristimo dugme *Snimi* a za povratak nazad dugme *Nazad*.

Slika 7. Forma za unos ulaznog naloga

Korisitimo dve tabele za smeštanje podataka :

Tabela Ulaz (opšti podaci naloga) ima sledeća polja :

Ime polja	Tip	Opis polja
<i>Broj</i>	Character 15	šifra naloga
<i>Datum</i>	Date	Datum stvaranja naloga
<i>Datum_pl</i>	Date	Datum kada nalog stiže za naplatu

<i>Naziv</i>	Character 15	Ime dobavljača
--------------	-----------------	----------------

Tabela Ulaz1 (stavke naloga) ima sledeća polja :

Ime polja	Tip	Opis polja
<i>Broj</i>	Character 15	šifra naloga
<i>ImeRobe</i>	Character 30	Naziv artikla
<i>NabCena</i>	Number	Nabavna cena artikla
<i>Količina</i>	Number	Količina koju smo kupili

5.2.3. Forma za pregled zaduženja

Ova forma nam prikazuje trenutno stanje dugovanja/potraživanja od dobavljača i kupca. Klikom na kružić pored dobavljača, listamo dobavljače, a na kružić pored kupca, listamo kupce. Izaberemo nekog dobavljača/kupca, period vremena koji nas interesuje (Od – Do) i pritisnemo na dugme *Pokaži*.

Ako hoćemo da vratimo dug, u donjem desnom uglu unesemo potrebne podatke i pritisnemo dugme *Snimi* i upisaćemo u tabelu zadati iznos. Sve ovo možemo i da odštampano pritiskom na *Print*. Pri dnu forme imamo skupa (sumirana polja) duguje i potražuje za dati period vremena.

The screenshot shows a software window titled "Kartica - Libra". It contains a table with columns: Datum, Broj naloga, Duguje, Potrazuje, and Naziv firme. The first row shows data for 10/01/2001, nalog number 1, with a debt of 16950 and a company name of FIKA TRADE. To the right of the table is a control panel. It has a dropdown menu for "Kupci" (currently showing "FIKA TRADE"), two radio buttons for "Dobavljači" and "Kupci" (with "Kupci" selected), and date pickers for "Od:" (12/12/00) and "Do:" (1/12/01). A "Pokaži" button is below these. Further down is a "Datum" picker (1/12/01), a label "Uplata /iznos/" with a text input field containing "0", a "Broj naloga" label with another text input field, and a "Snimi uplatu" button. At the bottom of the control panel are two large buttons: a pink "PRINT" button and a red "IZLAZ" button with an arrow. At the very bottom of the window, there are three summary fields: "Skupa duguje:" with the value 16950.00, "Skupa potrazuje:" with the value 0.00, and "Razlika:" with the value 16950.00.

Datum	Broj naloga	Duguje	Potrazuje	Naziv firme
10/01/2001	1	16950	0	FIKA TRADE

Kupci
FIKA TRADE

☐ Dobavljači
☒ Kupci

Od : 12/12/00 Do: 1/12/01

Pokaži

Datum
1/12/01

Uplata /iznos/
0

Broj naloga

Snimi uplatu

PRINT IZLAZ

Skupa duguje: 16950.00 Skupa potrazuje: 0.00 Razlika: 16950.00

Na sledećoj strani sledi deo koda koji formira SQL upit za prikaz podataka. Ovaj deo koda se izvršava kada pritisnemo dugme *Pokaži*.

```

procedure TfKartica.BtnPokaziClick(Sender: TObject);
var s,s1,s2,s3 : string;
begin
s:="";s1:="";
ComboBoxDobavljac.Text:=Trim(ComboBoxDobavljac.Text);
if RBDobavljaci.Checked then s:='KarticaD' else s:='KarticaK';// da li
radimo sa dobavljačima ili kupcima
ShortDateFormat:='mm/dd/yyyy'; // datum u bazi se nalazi u obliku
mm/dd/gggg
s1:=' and datum>= '+'""+DateToStr(DatumOd.date)+'""+' and datum<=
+'""+DateToStr(DatumDo.date)+'""';
ShortDateFormat:='dd/mm/yyyy';
with Baze.Query2 do
begin //formiraj SQL upit
SQL.Clear; //obriši ako je nešto bilo upisano
SQL.Add('select datum, broj, iznosD, iznosP, SUM(IznosD) as D,
SUM(IznosP) as P');
SQL.Add('from '+s);
SQL.Add('where naziv='+""+ComboBoxDobavljac.Text+""+s1);
SQL.Add('order by datum'); // da bude sortirano po datumu
end;
Open;
end;
end;

```

Za podatke koje vidimo na formi SQL upit bi izgledao :

```

Select datum,broj, iznosD, iznosP, Sum(IznosD) as D, Sum(IznosP) as P
from KarticaK
where naziv='FIKA TRADE' and datum>='12/12/2000' and
datum<='01/12/2001';

```

Upit *Query2* (sa njim stupamo u vezu sa bazom *KarticaK*) je povezan sa komponentom *DataSource_Query2* (povezuje pristup bazi sa komponentama za vizuelni prikaz), a ova komponenta je povezana sa *Database Grid* (tabela koju vidimo na formi) komponentom preko svojsva *DataSource* i rezultat našeg upita vidimo na ekranu.

Tabela *KarticaK* (odnosi se na Kupce) sadrži sledeća polja :

Ime polja	Tip	Opis polja
<i>Datum</i>	Date	Datum zaduživanja
<i>Naziv</i>	Character 30	Ime kupca
<i>IznosD</i>	Number	Dugovanje kupca
<i>IznosP</i>	Number	Potraživanje kupca
<i>Broj</i>	Character 10	šifra naloga na koji se odnosi dugovanje/potraživanje

Tabela KarticaD (odnosi se na dobavljače) je istovetna tabeli KarticaK (samo polje *Naziv* predstavlja ime dobavljača). Ne postoji primarni ključ, jer je moguće da se neki dobavljač zaduži dnevno više puta za robu.

6. ŠTA JE POTREBNO ZA DISTRIBUIRANJE APLIKACIJE?

Sve što je potrebno za distribuiranje jedne Delphi aplikacije je .EXE datoteka, direktorijum u kome se nalazi baza podataka, i instaliran BDE na odredišnoj mašini.

Što se tiče pravljenja instalacije programa, uz Delphi se dobija program *InstallShield Express* pomoću kojeg možemo napraviti instalaciju programa i uključiti i BDE. InstallShield zna koje datoteke su potrebne za instalaciju BDE-a, pa je taj deo posla automatski, a mi samo treba da uključimo .EXE fajl i direktorijum sa bazom podataka i definišemo alijas 'Diplomski' koji će pokazivati na bazu podataka. Nakon toga imamo napravljen program *Setup.exe* i još par datoteka koji nam predstavlja instalaciju našeg programa.

7. ZAKLJUČAK

U prvom delu rada je dat kratak opis programskog jezika Delphi, što je uvodno poglavlje za drugi deo rada koji se odnosi na rad sa bazama podataka. Kod proučavanja drugog dela rada potrebno je posedovati odgovarajuće znanje, kako o programskom jeziku Delphi i pisanju prostijih aplikacija u njemu, tako i o objektnom programiranju uopšte.

Praktična implementacija je izrađena u Delphi 5.0 razvojnom okruženju, koje se veoma dobro pokazalo za realizaciju ove aplikacije.

Model ove aplikacije je primenjiv kod realizacije mnogih problema koji se oslanjaju na rad sa lokalnim bazama podataka čiji su podaci tabelarno organizovani, a to je najčešće kod vođenja nekih evidencija.

Proširenje ovog modela programa može se ostvariti dodatnim procedurama i protokolima koji bi omogućili komunikaciju sa udaljenim serverom na kome je smeštena baza podataka.

što se tiče zauzeća mesta na hard disku istestirao sam program i to tako što sam uneo 10000 ulaznih i 10000 izlaznih naloga u proseku sa 4 stavke po nalogu i dobio sledeći rezultat :

Ime tabele	Broj slogova u tabeli	Zauzeće na hard disku
<i>Ulaz</i>	10,000	1.484,00 KB
<i>Ulaz1</i>	40,000	2.860,00 KB
<i>Izlaz</i>	10,000	1.484,00 KB
<i>Izlaz1</i>	40,000	2.860,00 KB
		8.688,00 KB

Sve ostale tabele zauzimaju mnogo manje mesta na disku.

Koristeći program za arhiviranje WinRar, spakovao sam tabele i dobio datoteku veličine : 416 KB. Znači kompresovali smo datoteke veličine 8,7 MB na 400KB, što je kompresija od oko 20 puta i čini backup baze podataka vrlo pogodnim (čak možemo i disketu koristiti za backup baza podataka).

Delphi je izuzetno jak programski jezik za razvoj programa koji rade sa bazama podataka. Moguće je koristiti kako za rad sa lokalnim bazama podataka, tako i za rad sa bazama na udaljenim serverima. Brzina rada sa bazama je odlična, zahvaljujući BDE-u. Pored toga ne postoji ni jedan problem koji se ne može rešiti u Delphiju. Može se pisati i čist API kod ako je potrebno, presretati poruke Windows operativnog sistema pre Delhpija i sami ih obrađivati, koristiti ActiveX kontrole, DLL datoteke, i sve ostalo što se koristi u drugim jezicima.

Kod nas se na primer Delphi koristi za rad sa evidencijom članova na PTT provajderu, u kombinaciji sa ORACLE-om.

Izuzetno je lak za početnike koji se prvi put sreću sa Windows programiranjem. Znači u Delphiju možete uraditi šta god Vam padne na pamet i to za mnogo kraće vreme nego u drugim programskim jezicima. Ja ga toplo preporučujem svima.

8. Literatura

- Osnovi računarske tehnike, Dr. @ivko Tošić, čuperak plavi, Niš 1994.g.
- Programski jezik Pascal, Dušan Tošić, Ivan Stojmenović, Naučna knjiga, Beograd 1989.g.
- Delphi punom snagom, Charles Calvert, Computer Equipment and Trade, Beograd
- Naučite Delphi za 21 dan, Kent Reisdorph, Kompjuter biblioteka, čačak, 1999.g.
- Delphi 2 unleashed, grupa autora, Sams publishing, United States,
- Foundations of Delphi development for Windows 95
- www.borland.com
- www.cabb.com/dd
- www.cs.monash.edu.au/vtran
- *e-mail*-ovi koja sam dobio od raznih ljudi koji se bave programiranjem u Delphiju